

Countering of Block Withholding for Miners

Connor Gardiner*, Loc Su†

Dept. of Computer Science, University of New Mexico
Albuquerque, NM

* cgardiner@unm.edu, † locsu@unm.edu

Abstract—In the mining world, the concept of mining pools have gained wide attentions as it has become an essential in blockchain’s Proof of Work. It allows individual miners to work together and combine their computational resources(in order to be more competitive and reduce the variance in their rewards) to perform quicker and with more efficiency has calculation to obtain wanted solution to generate blocks and gain rewards. This concept incentivizes small miners with less computational power to participate in the mining process, however, (the concept)(it also) introduces centralization risks and dependency on the pool operator. This trust dependency opens the door to multiple malicious attacks such as the block withholding (BWH). Block withholding attacks allow dishonest miners to increase their own profit by withholding any valid solutions from the mining pool. This increases the probability of the dishonest miner finding a block solo mining while simultaneously benefiting from the short term rewards associated with pooled mining. It is important to note that this type of behavior reduces the profitability of the mining pool. If this persists the pool operator will have to charge higher fees in order to compensate, which reduces the profitability of the honest miners mining to that pool.

In this paper, we examine a novel block withholding countermeasure for honest miners to maximize their own profit in the presence of a block withholding attack. The primary goal of which is to minimize the losses incurred by honest miners and the secondary goal is to minimize the profitability of block withholding attacks by increasing the competition for the increased profit associated with block withholding attacks. Inspired by the classic selfish mining variant of a block withholding attack, this method allows honest miners to allocates their resources between pool mining and solo mining to maximize their profit. Although this strategy is typically used by the BWH attacker, and requires information about the magnitude of the attack in order to optimize, we hope to demonstrate how this information can be used by honest miners to maximize

their own profit. From there we propose a method by which to measure the magnitude of a block withholding attack. Through probabilistic analysis and simulation, we can discover whether this technique can improve earnings under realistic network conditions, and we discuss its implications for miner behavior, and potentially disincentivize dishonest miners from malicious behaviors.

I. INTRODUCTION

A. Background

On January 3rd 2009, Satoshi Nakamoto presented Bitcoin, a peer-to peer network for value exchanges [1]. Throughout the years, with the rise of cryptocurrency, Bitcoin has been continuously breaking barriers with its application and consensus mechanism (PoW). PoW or Proof of Work is the consensus mechanism used in the Bitcoin blockchain network to validate and secure the network. At first glance, bitcoin mining may appear to be a great source of income, but due to the difficulty of hash calculation and valid block discovery, the variance in rewards is extreme. This makes it inaccessible to anyone that doesn’t have access to significant computational power. In order to solve this problem and create a more consistent payout, most miners decide to join mining pools. By joining a mining pool a miner accepts a lower expected reward in exchange for lower variance in that reward. Simultaneously the pool operator accepts the higher variance in their reward in exchange for an increased expected reward. The pool operator orchestrates the process of coordinating the computational power of the miners, and distributing rewards among them.

While the concept of mining pools propose more consistent income across the field, it introduces centralization risks and introduce incentive-based attacks such as Block Withholding attack. A block withholding

attack is an attack that has been known since the start of Bitcoin where a miner joins a mining pool and only submits their partial proof of work in order to damage the pool's profitability [2]. This not only reduces the collective reward but also forces honest miners to subsidize the dishonest miners' gains. This is extremely harmful toward the honest miners especially in small pools with high block finding variance.

This project addresses the challenge of how honest miners protect themselves from these malicious activities within mining pools. Throughout the past decade, we observed that there are many traditional responses that mainly focus on the detection mechanism or decentralized concept alternative. Unfortunately, those solutions are proven to either be slow or induce significant scalability and incentive challenges. We will further go into this in section II.

On the contrary, instead of focusing on the pool-level strategy, our paper explores a miner level strategy to minimize the damage of BWH while simultaneously increasing the honest miners profitability to the point where they can compete with the dishonest ones.

B. Motivation

While centralized mining pools offer consistent payouts and a reduction in reward variance for small miners, it also comes with many centralized risks. The mining pool concept is trust dependent which makes them vulnerable to malicious attacks such as block withholding. An honest miner who does not engage in attacks themselves will suffer reduced profitability by participating in a mining pool that is subject to a block withholding attack. The excess profit realized by the dishonest miners come from a corresponding decrease in the profits of either or both the honest miners or the pool operator. If they do not want to participate in pooled mining to avoid those attacks, the only option is to become solo miners. Though solo mining is not subject to the same risks as mining pools, most miners do not have a sufficient amount of computational hash power in their system to make a sustainable and consistent income.

This paper is motivated by the idea on how we can enable honest miners to act adversarially against dishonest miners by adapting their behavior dynamically,

specifically, by allowing them to decide the percentage of computing resources they want to allocate between pool mining and solo mining. This approach is designed to:

- Reduce the negative impact of dishonest activity within the pools
- Take advantage of favorable solo mining conditions (Opportunistic solo mining).
- Decrease dependence on centralized pools.

If such strategy exists, we can:

- Maintain honest miners' profitability under compromised circumstances.
- Dis-incentivize dishonest behavior.
- Encourage honest miners to remain honest, and ideally, incentivize dishonest miners to become honest.

II. RELATED WORKS

In the past, several research efforts have analyzed the vulnerabilities of mining pools while address some of the possible detection and prevention strategies against Block Withholding/Pool Hopping attacks. In this section, we will conduct analysis on various related studies done to address the Block Withholding attack. We will discuss different detection strategies and prevention methods while also highlighting the strengths and limitations of these approaches.

A. Two-Phase Proof Of Work.[3]

Rosenfeld and Eyal were among the first to address this issue by introducing "two-phase proof-of-work (PoW)" schemes. The approach is based on the idea that BWH attacks are possible due to miners' ability to distinguish the differences between partial and full proofs of work. Partial PoW (pPoW) and Full PoW (fPoW) are considered the two steps of PoW, hence, the name two-phase PoW. Rosenfeld's approach introduces three additional block fields—SecretSeed, ExtraHash, and SecretHash—which prohibited miners' ability to check the found block was a valid block or not.

- SecretSeed: The secret value that is only known to the pool operator
- ExtraHash: SHA256. The hash of SecretSeed distributed among the miners that allow them to verify the validity of partial work.

- **SecretHash**: the hash that used by the pool operator to validate the fPoW.

Miners only receive ExtraHash, preventing them from verifying whether a block meets the criteria for fPoW, while the pool manager uses SecretSeed to compute SecretHash and verify validity.

Eyal and Sirer also have the same approach but their method takes a different path. Their methods requires two conditions: the double-hash of the block header to be below the mining standard difficulty threshold and the header/the hash has to also be below the secret difficulty constraint (SecretHash). What sets this method apart from Rosen's is that it requires a private key along with its signature to separate the PoW process into two-phase.

B. Additional reward for the winning block [4]

Unlike the approach above, Bag and Sakurai's idea comes from the miners' ability to get their reward by just submitting partial proofs to the pool. A BWH attacker, despite withholding full PoW, will submit partial proofs and receive partial rewards for doing so. In order to counter this, they propose a scheme where there is a "special reward" for those who successfully submit a full proof of work coming from the pool's earning fund. With this scheme, since attackers tend to withhold full proofs, they would miss out on the "special reward" which damage their overall possible earning, hence, discourage them to perform such attack.

C. PMBWA [5]

In opposition to the two proposed methods above, this paper introduces a Prevention Method of Block Withholding Attack (PMBWA) that relies on analyzing miners' behavior within the blockchain network. The method works in several steps:

- **Data pre-processing**: perform data cleaning and verification by using a box chart detection algorithm.
- **Behavior evaluation**: evaluating three aspects of the miners' behavior; reward mechanism (reward honest behavior), punishment mechanism (punish malicious behavior), and credit model (reputation score attach to each miner).
- **Detection scheme**: through the Credit Level Classification Algorithm, they tracked the miners' behavior, conduct a posterior probability to understand whether the action is malicious or not and how closely it matches known good or bad patterns.
- **Income distribution**: After the classification process, they assign a weight variable to every miner. Those with high credits, denote that they are honest miners, they will get a higher reward, while miners with low credit get lower rewards.

III. METHODOLOGY

In this section we detail the development of our model of block withholding in the Bitcoin network.

A. Assumptions

These are our assumption on our simulation model:

- 1 mining pool using basic PPS payment scheme
- We allow the mining pool an infinite balance
- We assume knowledge of the amount of computational power controlled by dishonest miners
- The network aggregate profit is maximized when all the miners act optimally.

B. Variables

Before we get into the description, here are some of the important variables and their definitions used to define our functions:

- H : the Hash function.
- f : the pool fee
- x : percentage pool mining
- $1 - x$: percentage solo mining
- d : Percentage of dishonest miners.
- h : Percentage of honest miners.
- c : Scalar that represent the ratio between α δ and .
- δ : Scaled network difficulty
- λ : Real network difficulty.
- α : $H: \{1, ..., 2^\alpha\}$
- n : Network hash rate
- w : Percentage of block withholding
- b : Block reward
- t : block time in seconds
- k : Number of hashes computed by solo miners during a block.
- l : an estimator of $\frac{\delta}{2^\alpha}$

C. Simulation model description

```
class miner:
    def __init__(self, hashrate, honest,
        aggression):
        self.profit = 0
        self.hashrate = hashrate
        self.honest = honest
        self.aggression = aggression

    def print(self):
        print("hashrate:{} honest:{}
            aggression:{} profit:{}".format(
                self.hashrate, self.honest, self.
                aggression, self.profit))
```

Listing 1: Miner class definition

```
import random
'''valid shares is a list of all of the
    shares - those associated with block
    withholding'''
def get_winner(hxminers):
    indices = [x for x in range(len(
        hxminers))]
    return random.choices( indices,
        hxminers )
```

Listing 2: Blockchain network definition

```
class pool:
    def __init__(self, reserve):
        self.reserve = reserve
        self.totalhashrate=0
        self.realhashrate= 0
        self.fee = 0.06
        self.hashprice = 0

    def get_hashprice(self, nethx,
        blockreward):
        return ((1-self.fee) * blockreward)/
            nethx

    def mine(self,nethx, hx, honest,
        blockreward):
        self.totalhashrate+= hx
        self.realhashrate+= hx*honest
        pay = hx*self.get_hashprice(nethx,
            blockreward)
        self.reserve -= pay
```

```
return pay
```

Listing 3: Pool class definition

We begin by defining the aggregate expected profit of all miners as a function of $x = \frac{\text{poolhashrate}}{n}$. In order to analyze this we make the simplifying assumption that all miners allocate their work between pooled mining and solo mining identically. The aggregate expected profit function is constructed as follows: We use $\mathbf{B}$ to denote the block reward, and $\mathbf{f}$ the fee charged by the pool. The mining pool uses the PPS model. PPS defines the price of a hash as follows:

$$\text{hashprice} = \frac{B * (1 - f)}{n}$$

The expected profit of solo mining per hash is:

$$\frac{B}{n(1 - w)}$$

From which we obtain the function by normalizing to the network hashrate:

$$P(x) = B((1 - f)x + \frac{1 - x}{1 - w})$$

The motivation for considering the normalized form of the function above is that we can then create the corresponding profit function for the dishonest miners in aggregate, where we make the assumption that all of the pooled mining that they do is part of a block withholding attack, ie:

$$w = dx$$

and

$$P_d(x) = B((1 - f)dx + \frac{d(1 - x)}{1 - dx})$$

where $\mathbf{x}$ is the percentage of work allocated to pooled mining.

We can now maximize $P_d(x)$ by finding the root of it's derivative with respect to $\mathbf{x}$.

$$P'_d(x) = d(1 - f) + \frac{d(d - 1)}{(1 - dx)^2}$$

The corresponding root finding problem,

$$0 = d(1 - f) + \frac{d(d - 1)}{(1 - dx)^2}$$

Then by our assumption that the network aggregate profit is maximized when all miners allocate work identically, a value $\mathbf{x}$ that satisfies the root finding problem maximizes profit for all miners.

This describes the optimal strategy for an individual. The strategy is simply what percentage of resources one allocates to pooled mining, where the remaining resources are allocated to solo mining.

It is unrealistic to assume that all participants will act optimally, so we introduce a strategy parameter that describes how an individual's behavior deviates from the optimum. We call this parameter aggression, and it denotes the difference between the strategy employed by an individual and the optimal strategy. Negative values represent a preference for pooled mining, whereas positive values represent a preference for solo mining. Let a be the aggression value of some individual, i, $a \in [-1, 1]$

The individual allocates their resources according to the following nonlinear function, where x_i is the percentage of work that the individual allocates to pooled mining, and x is the observed fraction of pooled mining.

$$x_i = \max(0, \min(1, x - a))$$

st $x_i \in [0, 1]$

D. Block Withholding Rate Simulation

In our simulation above, we are operating under the assumption that the percentage of dishonest miners is known. With this information, we are able to calculate the block withholding rate, which serves as a key parameter in our method model. Unfortunately, in the real world, this information is often unachievable or difficult to estimate, hence, we develop a method for estimating the block withholding rate in real time without the prior knowledge of the distribution of honest and dishonest miners.

1) Scaling Argument Method: One of the tools we used to develop this estimation is the scaling method argument. The scaling method argument is essentially a technique that leverages the proportional relationship between the given input that is computable and the input that is almost impossible to compute. With this method, we can scale these observations to estimate the impact of unknown or variable factors. This allows us to approximate the block withholding rate under different assumptions.

Initially, we have two indicator r.v for solo and pool mining:

$$\text{Solo} = \begin{cases} 1 & \text{if solo miner wins block} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

$$\text{Pool} = \begin{cases} 1 & \text{if pool wins block} \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

With these random variables, we construct the probability of a solo miner finding a winning block:

$$P(\text{Solo} = 1) = \frac{1 - x}{1 - dx} \quad (3)$$

With our knowledge about the computational power associated with $1 - x$, corresponding to some number of hashes, denoted by k , computed during some time interval, we can show:

$$k = nt(1 - x) \quad (4)$$

We can also express the probability of finding a valid solution:

$$P(H \leq \lambda) = \frac{\lambda}{2^\alpha} \quad (5)$$

Hence, the probability of success in k hashes is:

$$P = \frac{k\lambda}{2^\alpha} \quad (6)$$

Furthermore, with (3) and (6), we can derive this:

$$\frac{k\lambda}{2^\alpha} = \frac{1 - x}{1 - dx} \quad (7)$$

Apply the same process for δ , we have

$$P(H \leq \delta) = \frac{\delta}{2^\alpha} \quad (8)$$

Let say $\exists c$ such that $\lambda = c\delta$, hence, $c = \frac{\lambda}{\delta}$ We can say

$$P(H \leq \lambda) = \frac{c\delta}{2^\alpha} \quad (9)$$

And finally with 6, 9 and 7, and :

$$\frac{kc\delta}{2^\alpha} = \frac{1 - x}{1 - dx} \quad (10)$$

Thus:

$$\frac{c\delta nt(1 - x)}{2^\alpha} = \frac{1 - x}{1 - dx} \quad (11)$$

2) **Emergent with multiples miners:** In order to simulate the scaling argument we use unsigned random ints and a random number generator to simulate sha256. In order to implement the mining race we discretize time by seconds and define a winning hash as the lowest hash computed in the one second interval during a block where a hash meets or exceeds the target difficulty. Let $\mathbf{L}$ be the indicator random variable that a hash meets or exceeds some difficulty target. Every miner samples $l = E(L)$ from their own solo mining hashes during each block and obtains a vector of l values with respect to a range of simulated difficulty targets. Then we unscale the $\mathbf{L}$ values and obtain the vector $\mathbf{y}$.

$$y = lc$$

At the end of the block the miners solve

$$w = 1 - \frac{1}{E(y)nt}$$

We justify the use of the average as a least squares fit to a constant. This can be verified by computing the Moore-Penrose pseudo-inverse of a column matrix of all ones, as is the case when using least squares to fit a single coefficient.

Despite having high expectations for this method we immediately met with significant errors. The magnitude of which were orders of magnitude larger than the upper bound on the variable w , and included sign errors. We initially expected the error to be due to scaling before fitting, but in our investigation we found that we had been mishandled our model. This prompted us to revisit our initial model and clarify its restrictions.

IV. RESULTS

As for the result, we have Figure 1,

- Black line (concave down): network aggregate expected profit
- Blue dashed line: dishonest miner aggregate profit
- Green dashed line: honest miner aggregate profit
- Red line: linear interpolation of $P'_d(x)$
- Brown line: $P'_d(x)$
- Black line (concave up): analogous quadratic root finding problem

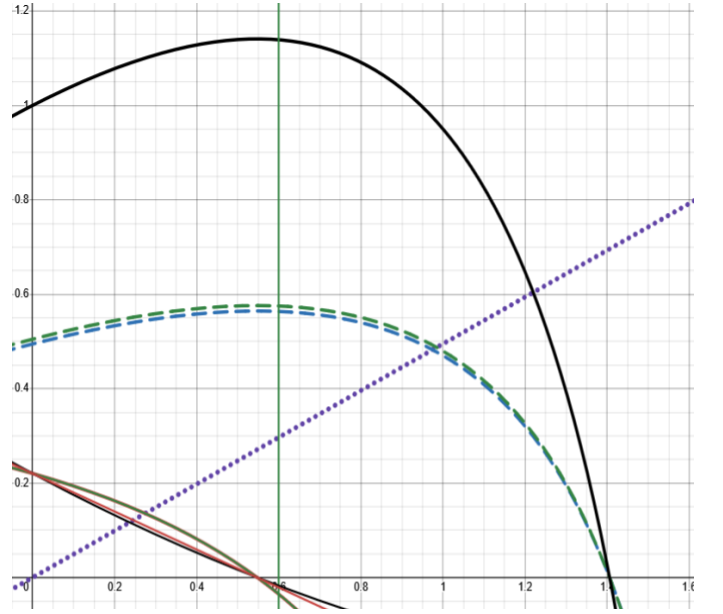


Fig. 1: Relationship between aggregate profit for different group of miners and the allocation of resources between pool mining and solo mining [6]

A. Bounds on Variables

After observing negative values of $\mathbf{x}$ and $\mathbf{w}$, which we expected to remain in the range $[0,1]$, it became apparent that we had to find the conditions that cause this.

1) **Upper bound on $\mathbf{d}$:** Consider the reformulation of the root finding problem presented earlier

$$0 = f + d - 2w(1 - f) + w^2(1 - f) \quad (12)$$

We can ensure $\mathbf{w}$ is not complex by adopting a constraint from the quadratic equation. That being that the square root of $b^2 - 4ac$ is a real number.

$$0 \leq (-2(1 - f))^2 - 4(1 - f)(f + d)$$

From this equation we obtain the bounds

$$d \leq 1 - 2f \quad (13)$$

$$w \leq x(1 - 2f) \quad (14)$$

2) **Pool Operator Profitability bounds::** If one has knowledge about the profitability of the pool operator then the following bounds can be inferred. note: Although the rest of our work treats $\mathbf{d}$ and $\mathbf{h}$ as unknown constants deriving these bounds requires us to treat $\mathbf{d}$ and $\mathbf{h}$ as random variables. This allows us to compare expected value of $\mathbf{h}$ and $\mathbf{d}$ to the expected profitability of the pool operator. If we infer the probability of the

mining pool finding a block from the profit equation we get

$$P(\text{pool} = 1) = (1 - f)x$$

and if we consider that only honest miners submit valid solutions to the mining pool we get a similar equation.

$$P(\text{pool} = 1) = hx$$

$$\begin{cases} hx < (1 - f)x & \text{if the pool operator shows a loss} \\ hx = (1 - f)x & \text{if the pool operator breaks even} \\ hx > (1 - f)x & \text{if the pool operator shows a profit} \end{cases} \quad (15)$$

Recall $h = 1 - d$

$$\begin{cases} d > f & \text{if the pool operator shows a loss} \\ d = f & \text{if the pool operator breaks even} \\ d < f & \text{if the pool operator shows a profit} \end{cases} \quad (16)$$

V. APPROXIMATION OF $P'_d(x)$

The root of $P'_d(x)$ is somewhat unwieldy because solving for it by the quadratic formula can result in complex roots. One way to circumvent this is by deriving a line that intersects the x and y intercept of $P'_d(x)$. recall the root finding problem

$$0 = d(1 - f) + \frac{d(d - 1)}{(1 - dx)^2}$$

we can manipulate this into an analogous root finding problem

$$0 = d(1 - f)(1 - dx)^2 + d(d - 1)$$

which corresponds to a simpler quadratic in x that shares x and y intercepts with the $P'_d(x)$

$$y = d(1 - f)(1 - dx)^2 + d(d - 1)$$

This quadratic has an **x intercept** at the point

$$\left(\frac{1}{d}\left(1 - \left(\frac{1-d}{1-f}\right)^{1/2}\right), 0\right)$$

and **y intercept** at

$$(0, d(d - f))$$

The line that interpolates both points is

$$\hat{P}'_d(x) = \frac{-d(d - f)x}{\frac{1}{d}\left(\frac{1-d}{1-f}\right)^{1/2}} + d(d - f) \quad (17)$$

Now we have a definition of the root which is always real,

$$\frac{1-d}{1-f} \geq 0$$

, and a reasonable approximation of the derivative at points near the root. Investigating applications of this interpolation beyond root finding is a potential area for future work.

VI. CONCLUSION

Although the primary goal was to investigate block withholding attacks through simulation, we prioritized developing a model robust enough to mirror the aspects of the problem we identified as being important. We reached a point in the middle of refactoring the simulation where it became apparent that we had significant errors. At this point we made the decision to revisit the theoretical aspects of our model in order to ensure that at least that aspect of our results could be presented error free. We were able to correct the necessary theoretical aspects of the model at the cost of fixing the simulation and obtaining simulation results. Despite falling short of our aims of demonstrating the potential for honest miners to reduce the profitability of dishonest miners performing block withholding attacks, we were able to show that honest miners can profit proportionally to a group of dishonest miners if they allocate their work in the same way. We made no significant leaps towards our goal but made significant strides developing the foundation for continued work.

VII. FUTURE WORK

While our work has shown theoretically that everyone can profit to the same degree from block withholding attacks, we provide no practical method for measuring the amount of block withholding present in the Bitcoin network or individual mining pools. Areas for future work include:

- Development of methods to approximate block withholding in real time.
- Substituting pool hopping for solo mining as a short term optimization for smaller miners.
- Accounting for Non-Optimal Miner Behavior in the theoretical model of aggregate profit.

A. Block withholding calculation issues

At the time of writing the real probability that we are trying to measure fluctuations about is on the order

of 10^{-64} . Making measurements of such small values with high accuracy is quite difficult, but there are several strategies that could be employed to improve the error rate. One such strategy is to compute the slope of the line that interpolates $(0, 0)$ and (δ, l) . Given many samples of such slopes one could take the average and fit to the slopes directly. Alternatively, one could consider a trimmed mean of the same slope values, about the true mean of 1. The true mean is obtained by noticing

$$P(H \leq 0) = 0$$

and

$$P(H \leq 2^\alpha) = 1$$

Yet another potential solution is to ignore all samples of slopes less than 1 based on the argument that block withholding can only increase the probability of finding a valid block. There are further improvements that could be made such as reordering the equation to avoid dividing by very large numbers as follows:

$$\left(\sum^{N(L)} L \right) \lambda n t (1 - w) = N(L) \delta$$

and

$$\lambda n t (1 - w) = 2^\alpha$$

B. Pool hopping for solo miners along side with credit based model and "special reward" scheme

Normally, the concept of pool hopping usually have a negative connotation in which miners are freely switching between different mining pools in order to maximize expected rewards [7]. As for our approach, unlike the traditional ways, we have solo miners alternate between solo mining and pool mining. This strategy allows them to have the chance to generate more revenue.

Furthermore, we can expand on this idea by applying the concept of credit model and the "special reward" scheme from the Related Work section. By combining the PMBWA's credit evaluation with our solo/pool hopping model. Solo miners can decide which pool to hop into based on the overall credit score of the pool. Additionally, with "special reward" mechanism, the miners know that if a pool offers bonus reward for a full solution submission, it is less infected by the BWH attackers which will offer more incentive for them.

Solo miners could prioritize these pools in their hopping decision making.

C. Accounting for Non-Optimal Miner Behavior

With our current model, we make the assumption that all miners behave optimally through making rational set of decisions, aiming to maximize their expected profit. While this assumption ease our difficulty level, it does not fully reflect the real-world mining behavior. As part of future work, we plan to extend our model to more realistic scenarios in which only a subset of miners act optimally (dishonest miners), while others can act sub-optimally (honest miners). This would allow us to begin analyzing how miner behavior impacts profitability of block withholding, its interaction with network aggregate profit and distribution of block rewards. The hope being that honest miners could decrease the profitability of block withholding attacks by modifying their own behavior in response to network conditions.

VIII. REFERENCES

REFERENCES

- [1] Yean-Fu Wen and Ming-Fang Chen. “Mined Block Withholding and Imposed Fork by Using Mining Pool Alliance Strategic—A Case Study in Bitcoin System”. In: *IEEE Access* PP (Jan. 2024), pp. 1–1. DOI: 10.1109/ACCESS.2024.3522962.
- [2] Samiran Bag, Sushmita Ruj, and Kouichi Sakurai. “Bitcoin Block Withholding Attack: Analysis and Mitigation”. In: *IEEE Transactions on Information Forensics and Security* 12.8 (2017), pp. 1967–1978. DOI: 10.1109/TIFS.2016.2623588.
- [3] Suhyeon Lee and Seungjoo Kim. *Countering Block Withholding Attack Efficiently*. Cryptology ePrint Archive, Report 2018/1211. 2018. URL: <https://eprint.iacr.org/2018/1211>.
- [4] Samiran Bag and Kouichi Sakurai. “Yet Another Note on Block Withholding Attack on Bitcoin Mining Pools”. In: *Information Security (ISC 2016)*. Vol. 9866. Lecture Notes in Computer Science. Springer, 2016, pp. 167–180. DOI: 10.1007/978-3-319-45871-7_11. URL: https://link.springer.com/chapter/10.1007/978-3-319-45871-7_11.
- [5] Hao Chen et al. “Prevention method of block withholding attack based on miners’ mining behavior in blockchain”. In: *Applied Intelligence* 53 (2023), pp. 9878–9896. DOI: 10.1007/s10489-022-03889-3. URL: <https://link.springer.com/article/10.1007/s10489-022-03889-3>.
- [6] Desmos Inc. *Desmos Graphing Calculator*. Accessed: 2025-05-15. 2024. URL: <https://www.desmos.com/calculator/wjlyskds4i>.
- [7] Natkamon Tovanich et al. “An Empirical Analysis of Pool Hopping Behavior in the Bitcoin Blockchain”. In: *2021 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. 2021, pp. 1–9. DOI: 10.1109/ICBC51069.2021.9461118.